Playing Catan on a one-second budget

Team larpmaxxers · Catan agent hackathon · 17 October 2026

Our final agent wins 76.1% of its games against the organisers' bot F and 54.0% against their trained bot O (1,000 public games each, 228 ms of CPU per game on average). It is F's own evaluation function with six small additions, each tested on identical games and kept only after it held up on fresh boards. This report explains why the clock leaves little room for search, what each addition does, what failed, and why O is still ahead.

1 The clock decides the design

An agent gets 300 ms of CPU at the start of a game and 12.5 ms more for each decision it is asked to make, up to 1.5 s. A game asks about 56 times, so the average allowance is one second. On this engine a candidate move costs one copy of the game, one execution of the move and one call to the evaluation function, between 0.2 and 0.3 ms depending on the machine. A decision can therefore afford fifty or sixty candidates. Move counts are small (median 5, ninetieth percentile 18, maximum 54), and 92% of the positions one move away are still our own turn, so a search that waits for the opponent's reply rarely gets to run.

This fixes the strategy. Looking deeper is paid for out of the same fifty evaluations, and it only helps if the evaluation can tell the deeper positions apart. Ours did not: searching deeper with F's own function scored about 0.50 against F in four separate attempts by the team, and our own two-step lookahead scored 0.492 ± 0.049 over 400 games. We spent the clock on a better evaluation instead, and on a few decisions where an exact calculation is cheap.

2 The opponent is a ladder

F scores every position it can reach in one move and plays the best. The score is a ladder of terms whose weights differ by orders of magnitude:

$$V(s) \;=\; 3{\cdot}10^{14}\,VP \;+\; 10^{8}\,(P_{me} - P_{opp}) \;+\; 10^{4}\,R \;+\; 10^{3}\,B \;+\; 10^{2}\,H \;+\; \dots$$

P is expected production, R the production reachable with one more road, B the number of legal settlement sites and H how close the hand is to a city or settlement. One *pip*, a 1 in 36 chance of a card per turn, equals $2.778{\cdot}10^{6}$ units. Each rung outweighs everything below it: F takes a point whenever it can, then fights for production, then for room to expand.

Two facts shaped our work. A new term changes F's choices only if it is priced in pips, since a term worth a hundred units is noise. And F is deterministic, so its opening placements and robber moves are functions of the board, which we can compute exactly. Its blind spots follow from the weights: a longer road is worth 0.1 units per segment until F has no site left to build on, holding more than seven cards costs 5 units against 2.8 million for a pip, and a knight card is worth about 10.

3 The agent, layer by layer

Each layer was added alone and measured against the version before it on identical games. Gains are in percentage points of win rate against F unless stated.

Two layers need more than a table row. **The opening solver** scores a finished layout of four settlements in pips: production with each resource weighted by its scarcity on this board, $(\text{mean}/\text{total})^{½}$; 4 pips for each distinct resource; a bonus for ports on resources we produce; credit for open sites two roads away; minus half of the rival's own score. For the twelve best first candidates it plays F's reply forward on a copy of the game, which is exact because F's opening is deterministic. It helps the first seat clearly and the second seat not at all (0.685 → 0.665); a joint search over the second seat's two back-to-back placements changed the pick in only 11 of 400 games.

The trade gate. A greedy agent takes a bank trade whenever the hand looks slightly better afterwards. Over 400 games a trade was the greedy choice 2,312 times and the gate vetoed 676 of them. The knight rule alone gained nothing (21 games to 22), so most of the pooled gain probably belongs to the gate.

Layer	What it does	Measured gain
Tail terms	+1 pip per segment of road lead over the rival, −1 pip per card above seven, +1 pip per knight	0.627 over 1,000 games; +11 to +12 over a copy of F
Opening solver	Scores whole starting layouts in pips and plays F's reply forward	+4.4 (p = 0.022), 1,000 games; first seat 0.545 → 0.685
Win search	From 8 VP, searches every sequence inside the turn for a guaranteed win	10 games won, 0 lost (p ≈ 0.002)
Robber rule	Puts the robber where it denies the rival most, pricing the steal from the rival's exact hand	+4.5 on the screening block (p = 0.054), then +1.5 and +1.75 on fresh blocks
Knight rule and trade gate	Knight played before the roll when it frees our best tile or takes the army. A bank trade is allowed only if it makes a city, settlement or development card affordable at once, or the hand exceeds seven	+3.75 and +2.5 on two fresh blocks; pooled 74 to 49 wins (p = 0.030)
Build readiness	+2 pips scaled by progress toward a city (wheat to 2, ore to 3), and +2 pips scaled by how many of wood, brick, sheep, wheat are held when a settlement site is open	+8.5 against our earlier agent (70/36, p = 0.001); +5.5 against F (57/35, p = 0.028)

4 How we measured

Catan outcomes are mostly luck. In F against F, 88.5% of boards are won by whichever seat moves first in *both* games of a pair, so two independent win rates at 400 games (±4.9 points) cannot separate agents that differ by two or three points. Every comparison was therefore paired: both agents play the same boards, seats and dice, and only the games they win differently carry information. Writing *b* for games the candidate won and the reference lost, and *c* for the reverse, an exact binomial test of *b* against *c* gives the p-value.

Four rules kept us honest. Tuning, confirmation, verification and reporting used four disjoint blocks of board seeds. A win had to repeat on a second block before we kept it, because we screened dozens of ideas and about one null idea in ten shows p < 0.1. Faults had to be exactly zero: an earlier agent with a fault rate of 0.97% lost 15.5 points while passing every other threshold. And every candidate also played at half clock speed with no game over budget, as insurance against a slower scoring machine.

5 Results

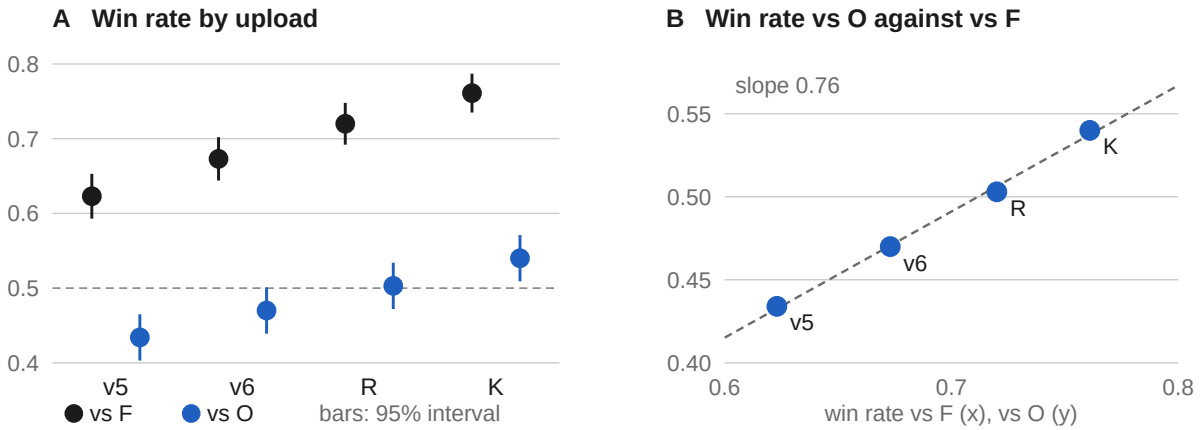


Figure 1. Scored uploads on the 1,000-game public set. (A) Both columns rose with every upload. (B) O plotted against F; the dashed line is a least-squares fit through four points and is descriptive only.

Upload	vs F	vs O	Mean	CPU per game, mean / worst
v5	0.623 ± 0.030	0.434 ± 0.031	0.529	183 / 511 ms
v6 (opening solver)	0.673 ± 0.029	0.470 ± 0.031	0.572	197 / 489 ms
R (win search, robber rule)	0.720 ± 0.028	0.503 ± 0.031	0.612	239 / 1,430 ms
K (R + knight, trade gate, readiness)	0.761 ± 0.026	0.540 ± 0.031	0.651	228 / 1,140 ms

All four scored with zero faults and no game over the clock. R's worst single game used 1,430 ms of the 1,500 ms cap, which is why we kept the clock rule. Each upload improved both columns, and the O column rose 0.76 points for every point gained against F. Changes that work against F therefore carried over to O at about three quarters strength, although none was designed with O in mind. O stays 0.19 to 0.22 below F in every upload. A fifth upload, with larger readiness weights, was still queued when we wrote this.

6 What did not work

Idea	Result against F
Deeper search with F's own function	About 0.50 in four attempts; our two-step version 0.492 ± 0.049
Weights fitted to outcomes (589,312 positions from 4,000 games)	Held-out AUC 0.832, yet the agents we screened with them scored 0.29 to 0.62 against 0.615 for the base agent
Reward for holding development cards	0.145 over 200 games: the agent hoards and never plays them
Turn planner chaining bank trades	0.61 against 0.72 for the same agent without it ($p < 0.001$)
Keep ore and wheat when a city is one card away (no development buy)	−6.0 points (31/55, $p = 0.013$)
Buy a development card whenever no point is affordable	Cut the knight-and-gate gain from +3.75 and +2.5 to +1.5 and −3.5
Chance-averaged card buys, road-building lookahead, second-seat opening search, endgame race term	−1.0 to +0.25 points, all within noise

The fitted evaluation is the instructive failure. An AUC of 0.83 says the fit ranks positions from different games well. A decision compares positions one move apart, which differ in one to three features, and the fit saw almost nothing there. Ranking positions and ranking moves are different measurements. The development-card results point the same way: a card in hand looks like value to a one-move evaluator, so a bonus for holding it makes the agent keep it.

7 Why O is still ahead

O is not available locally, so every O number comes from the portal and carries about ±0.03. The gap to F did not shrink as we improved (0.189, 0.203, 0.217, 0.221). Self-play gave one clue. Terms that did nothing against F became clearly useful against our own earlier agent: readiness at weight two with a knight cap of three scored 0.5375 against it where the reference scored 0.4625 (74 to 44, $p = 0.007$), while against F it moved only from 0.745 to 0.755. F does not punish a slow economy, so F cannot teach us to avoid one. Removing the opening simulation cost 0.02 against our own agent, so the opening's gain is not only an exploit of F's habits. We expect the next gains from training against our own agents and a pool of differently styled opponents, not from more tuning against F.

Limits. Each leaderboard figure is one 1,000-game measurement on public boards, and the private boards and group stage may differ. With four scored uploads we cannot separate the contribution of each layer against O; the layer table is measured against F and against our own earlier agent.

8 Lessons beyond this game

Spend a small budget on knowing, not on looking. With fifty evaluations per decision, depth bought nothing and a better description of the position bought several points.

A good ranking of positions is not a good ranking of moves. The fitted evaluation predicted winners well and chose moves badly, because a choice is made between near-identical positions.

Screening needs a second look. The robber rule gained 4.5 points on the block where we found it and about 1.6 on fresh ones. We trust a gain only after it appears again on fresh seeds.

Test against more than the opponent you are given. Once an agent beats its anchor by a wide margin, the anchor stops teaching it, and a copy of the agent itself shows the mistakes the anchor lets pass.

9 Public material and tools

We used catanatron (B. Collazo, GPL-3.0-or-later), pinned to the commit in *requirements.txt*, including its ValueFunctionPlayer, which is F. Fitting a linear evaluation to game outcomes follows Buro (2002), and the paired test is the exact binomial (McNemar) test on discordant games. A coding assistant (Claude Code) wrote and ran experiment code and helped analyse results, as section 11.3 of the rules allows. The submitted agent calls no model and uses no network. Every number above comes from a logged run in our repository.